

Principles of the FAT file system

—<http://www.sjhf.net>

Text structure index: Image table index:

- 1. The physical structure of the hard disk
- 2. The logical structure of the hard disk

- 3. Disk boot principle
 - 3.1 MBR sector
 - 3.2 Extend partitions
 - 4.1.1 FAT32DBR扇区
 - 4.1.2 FAT16DBR扇区
 - 4.1 About DBR

- 4. The principle of FAT partitioning
 - 4.2 About retaining sectors
 - 4.3 Storage principles of FAT tables and data

5. End

Figure 1 The physical structure of a hard drive
Figure 2 Disk MBR under winhex

Figure 3 The significance of the MBR parameters given by winhex
Figure 4 Partition table type flag

Figure 5 illustrates the disk structure of a 4-partition disk
Figure 6 Partition table link illustration

Figure 7 illustration of the overall structure of the disk
Figure 8 DBR diagram of FAT32 basic partition under winhex

Figure 9 Figure 8DBR parameter description given by winhex
Figure 10 File allocation table of FAT16 intercepted by winhex

Figure 4.3.11 Organization of FAT16
Figure 4.3.12 Organization of FAT32

Table 1 Partition table parameters

Table 2 Expanding the contents of a partition table key
Table 3 Location division of FAT32 partition DBR
Table 4 BPB fields for FAT32 partitions

Table 5 Extended BPB fields for FAT32 partitions
Table 6 DBR composition on FAT16 partitions

Table 7 BPB fields for FAT16 partitions
Table 8 Extended BPB fields for FAT16 partitions

Table 9 FAT16 partition size and pair factor cluster size

Table 11 FAT16 Definitions of Catalog Items

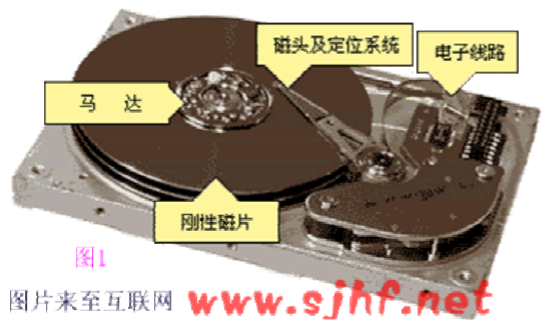
Table 12 FAT32 partition size and pair factor cluster size

Table 13 Meaning of FAT Table

Table 14 FAT32 Definitions of Short File Directory Items

Table 15 FAT32 Definitions of Long File Directory Items

1. Physical structure of hard disk:



The hard disk storage data is realized according to the principle of electrical and magnetic conversion. A hard disk consists of one or several material platters such as metal or glass coated with magnetic substances, as well as magnetic heads mounted on both sides of the platter and corresponding control circuitry (Figure 1), where the platters and heads are sealed in a dust-free metal shell. When the hard disk is working, the platter rotates at high speed at the design speed, and the magnetic head set on the surface of the platter moves radially to the specified position under the control of the circuit and stores or reads the data. When the system writes data to the hard disk, the magnetic field generated by the "write data" current in the head changes the state of the magnetic substance on the surface of the platter, and can still be maintained after the magnetic field of the write current disappears, so that the data is stored. When the system reads data from the hard disk

The magnetic head passes through the designated area of the platter, and the magnetic field on the surface of the platter causes the magnetic head to generate induced current or the impedance of the coil to change, and is restored to data after processing by the relevant circuit. Therefore, as long as the platter surface can be treated more smoothly, the head can be designed more precisely, and the platter rotation speed can be maximized, a hard drive with a larger capacity and faster data reading and writing speed can be created. This is because the flatter the surface treatment of the platter and the faster the rotation speed, the closer the head is to the surface of the platter, improving the reading and writing sensitivity and speed; The smaller and more precise the head design, the smaller the space the head occupies on the platter, so that the head builds more tracks on a platter to store more data.

[Back to index] 2. The logical structure of the hard disk.

A hard disk is made up of many platters, each of which has a read and write head on each side. If there are N platters. There are 2N faces, corresponding to 2N heads, numbered from 0, 1, 2. Each platter is divided into several concentric circuits (logically, invisible). The rules for dividing each platter are usually the same. In this way, the radius of each platter is a concentric circle with a fixed value R, and then logically forms a cylinder with the motor spindle as the axis, numbered 0, 1, and 2 from the outside to the inside..... Each track on each platter is divided into dozens of sectors, usually with a capacity of 512bytes, numbered 1, 2, 3 according to certain rules..... Cylinders×Heads×Sector sectors. These three parameters are the physical parameters of the hard disk. Many of our practices below require a deep understanding of the meaning of these three parameters.

[Back to Index] 3. Disk boot principle.

3.1 MBR (master boot record) sector: After pressing the power key, the computer starts executing the motherboard BIOS program. After a series of tests and configurations. Start booting the system in the system boot order set in the BIOS. Assume it's a hard drive now. How does the BIOS hand over the execution authority to the hard disk after executing its own program? After handing it over to the hard disk, it executes the program where it is stored. In fact, a piece of code called MBR plays a pivotal role. MBR (masterboot record), that is, the master boot record, sometimes called the main boot sector. Located in the entire hard disk 0 cylindrical 0 head 1 sector (which can be regarded as the first sector of the hard disk), the BIOS will jump to the first instruction in the MBR after executing its own program. Leave control of the system to MBR. In a total of 512 bytes of the main boot record, the MBR's bootloader occupies the first 446 bytes (offset 0H ~ offset 1BDH), the subsequent 64 bytes (offset 1BEH~offset 1FDH) are DPT (DiskPartitionTable), and the last two bytes "55 AA" (offset 1FEH~offset 1FFH) are the effective end of the partition. MBR does not vary from operating system to operating system, which means that different operating systems may have the same MBR, and even if it is different, MBR will not carry the nature of the operating system. It has the characteristics of public guidance. Let's analyze a piece of MBR. Below is an MBR of a Seagate 120GB hard drive viewed with winhex.

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	访问
0000000000	33	C0	8E	D0	BC	00	7C	FB	50	07	50	1F	FC	BE	1B	7C	3缺屑。 鵠.P. .
0000000010	BF	1B	06	50	57	B9	E5	01	F3	A4	CB	BD	BE	07	B1	04	?.PW瑰.螭私??
0000000020	38	6E	00	7C	09	75	13	83	C5	10	E2	F4	CD	18	8B	F5	8n. .u.廻.作?嫖
0000000030	83	C6	10	49	74	19	38	2C	74	F6	A0	B5	07	B4	07	8B	糖.It.8,t鰻??
0000000040	F0	AC	3C	00	74	FC	BB	07	00	B4	0E	CD	10	EB	F2	88	朕<.t ..??胸
0000000050	4E	10	E8	46	00	73	2A	FE	46	10	80	7E	04	0B	74	0B	N.鐵.s*樺.€~.t.
0000000060	80	7E	04	0C	74	05	A0	B6	07	75	D2	80	46	02	06	83	€~.t.柚.u襴F..
0000000070	46	08	06	83	56	0A	00	E8	21	00	73	05	A0	B6	07	EB	F..傳..?.s.柚.ẽ
0000000080	BC	81	3E	FE	7D	55	AA	74	0B	80	7E	10	00	74	C8	A0	紛>襴U簿.€~.t衰
0000000090	B7	07	EB	A9	8B	FC	1E	57	8B	F5	CB	BF	05	00	8A	56	?塋巖.w嫖丝..葵V
00000000A0	00	B4	08	CD	13	72	23	8A	C1	24	3F	98	8A	DE	8A	FC	.??r#姚\$?楨迄 \$u
00000000B0	43	F7	E3	8B	D1	86	D6	B1	06	D2	EE	42	F7	E2	39	56	C摩堯噫?翌B麾9VV
00000000C0	0A	77	23	72	05	39	46	08	73	1C	B8	01	02	BB	00	7C	.w#r.9F.s.?.? .
00000000D0	8B	4E	02	8B	56	00	CD	13	73	51	4F	74	4E	32	E4	8A	姚.媽.?sQ0tN2鑄S
00000000E0	56	00	CD	13	EB	E4	8A	56	00	60	BB	AA	55	B4	41	CD	V.?脰葵.`华U碯 í
00000000F0	13	72	36	81	FB	55	AA	75	30	F6	C1	01	74	2B	61	60	.r6伽須0隼.t+a`
0000000100	6A	00	6A	00	FF	76	0A	FF	76	08	6A	00	68	00	7C	6A	j.j. v. v.j.h.
0000000110	01	6A	10	B4	42	8B	F4	CD	13	61	61	73	0E	4F	74	0B	.j.磨燭?aa.s.Ot..
0000000120	32	E4	8A	56	00	CD	13	EB	D6	61	F9	C3	49	6E	76	61	2鑄V.?胚a Invaa
0000000130	6C	69	64	20	70	61	72	74	69	74	69	6F	6E	20	74	61	lid partition ta
0000000140	62	6C	65	00	45	72	72	6F	72	20	6C	6F	61	64	69	6E	ble.Error loadin
0000000150	67	20	6F	70	65	72	61	74	69	6E	67	20	73	79	73	74	g operating syst
0000000160	65	6D	00	4D	69	73	73	69	6E	67	20	6F	70	65	72	61	em.Missing opera
0000000170	74	69	6E	67	20	73	79	73	74	65	6D	00	00	00	00	00	ting system.....
0000000180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0000000190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	 MBR引导代码
00000001A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000001B0	00	00	00	00	00	2C	44	63	33	B1	33	B1	00	00	80	01	,Dc3???.€.
00000001C0	01	00	07	FE	FF	7B	3F	00	00	00	3D	A8	DA	00	00	00	 2个硬盘分区表
00000001D0	C1	7C	0F	FE	FF	7C	A8	DA	00	45	8F	1E	0D	00	00	00	 输? Y.E?...
00000001E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	 分区有效标志
00000001F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	 00

图2

[Back to Index]

Your hard drive's MBR boot code may not be like this. However, even if they are different, the functions performed are generally the same.

Let's look at the DPT section. The operating system is designed to facilitate the management of disks. The concept of disk partitioning has been added. That is, dividing a disk logic into several pieces. The number of disk partitions is limited by the number of letters in C^Z, how to represent the attributes of multiple partitions in the DPT in the above figure of 64 bytes? Microsoft solves this problem with a linked approach. In a DPT of 64 bytes, 16 bytes describe the properties of a partition in partition entry units. That is, the first partition table entry describes the properties of a partition, which is usually the basic partition. The second partition table entry describes the rest of the space except the basic partition, which is generally what we call an extended partition. A general description of this part is shown in Table 1.

Table 1 Figure 2 Partition

field	field byte	bits	Field name and
shift	length	value	
t			
			Guide indicator (BootIndicator)
0x01BE	BYTE	0x80	finger Indicate whether the partition is Active partition.
0x01BF	BYTE	0x01	开始磁头 (Starting Head)
			The StartingSector is used
0x01C0	6 bits	0x01	0~5 bits. The next two (6th and 6th 7) is started cylindrical field used
			开始柱面 (StartingCylinder) except
0x01C1	10 places	0x00	In addition to the last two digits of the Start Sector field, 1 bit is also used to make up the cylindrical value. The starting cylinder is a 10- digit number, the maximum is 1023
0x01C2	BYTE	0x07	The SystemID defines the type of partition and is detailed See Figure 4
0x01C3	BYTE	0xFE	Ending Head (Ending Head)
			Ending sector (Ending Sector) only
0x01C4	6 bits	0xFF	0~5 bits are used. The last two (6th and 7th) are finished on the cylinder field used
			Ending Cylinder (Ending cylinder) uses 1 bit in addition to the last two digits of the ending sector field to make up this cylindrical value. The ending cylinder is a 10- digit number, the largest The value is 1023
0x01C5	10 places	0x7B	
0x01C6	DWORD	0x0000003F	Relative sector number

(RelativeSecto

rs) 从该磁

The amount of displacement from the
beginning of the disk to the beginning
of the partition, measured in sectors

calculate

Total Sectors

0x01CA DWORD 0x00DAA83D The partition

The total number of sectors in the

[Back to Index] Note: The data in the above table is displayed as actual data, that is, in a high-to-position manner. It is stored from low to high position. The two behave differently, please read carefully. The tables that appear later are the same.

You can also see the meaning of these parameters in Winhex:

Master Boot Record, 基础偏移量: 0		
Offset	标题	数值
0 偏移	Master bootstrap loader code 引导代码	33 C0 8E D0 BC 00 7C FB
Partition Table Entry #1		
1BE	80 = active partition 活动分区标志	80 80表示活动, 00表示非
1BF	Start head 开始磁头	1
1C0	Start sector 开始扇区	1
1C0	Start cylinder 开始柱面	0
1C2	Operating system indicator (hex)	07 分区类型标志
1C3	End head 结束磁头	254
1C4	End sector 结束扇区	63
1C4	End cylinder 结束柱面	891
1C6	Sectors preceding partition 1	63 本分区之前的扇区数
1CA	Length of partition 1 in sector	14329917 本分区的扇区数
Partition Table Entry #2		
1CE	80 = active partition	00
1CF	Start head	0
1D0	Start sector	1
1D0	Start cylinder	892
1D2	Operating system indicator (hex)	0F
1D3	End head	254
1D4	End sector	63
1D4	End cylinder	1023
1D6	Sectors preceding partition 2	14329980
1DA	Length of partition 2 in sector	220106565
Partition Table Entry #3		
1DE 0H	80 = active partition	00
1DF 1H	Start head	0
1E0 2H	Start sector	0
1E0 2H	Start cylinder	0
1E2 4H	Operating system indicator (hex)	00
1E3 5H	End head	0
1E4 6H	End sector	0
1E4 6H	End cylinder	0
1E6 8H	Sectors preceding partition 3	0
1EA CH	Length of partition 3 in sector	0
Partition Table Entry #4		
1EE	80 = active partition	00
1EF	Start head	0
1F0	Start sector	0
1F0	Start cylinder	0
1F2	Operating system indicator (hex)	00
1F3	End head	0
1F4	End sector	0
1F4	End cylinder	0
1F6	Sectors preceding partition 4	0

[Back to Index]

Description: Each partition table key occupies 16 bytes, assuming the offset address starts at 0. Partition table entry 3 in Figure 3. Partition table item 4 is the same as partition table item 3.

1. Whether the 0H offset is the mark of the active partition can only be 00H and 80H. 80H is active and 00H is inactive. The rest of the values are illegal to Microsoft.

2. To reiterate (this is very important): numbers greater than 1 byte are stored in a small endian format or reverse byte order. The low-byte first format is a way to save numbers so that the lowest byte appears first in the hexadecimal number symbol. For example, a low byte of the value 0x3F000000 for a relative sector number field is preceded by 0x0000003F. The decimal number of the format number with this low byte is 63.

3. When the system is partitioned, each partition is not allowed to cross the cylindrical plane, that is, it is all measured by the cylindrical surface, which is commonly referred to as the partition granularity. Sometimes the size of the input partition is 7000M, but the partition is 6997M, which is the reason. In the sector and cylindrical parameters of offset 2H and offset 6H, the sector occupies 6 bits and the cylindrical surface occupies 10 bits. The two higher digits are the high two of the 10 cylindrical positions, and the 8 digits composed of offset 7H are the lower 8 digits of the 10 cylinder numbers. It can be seen that the partition capacity actually expressed in this way is limited, the cylindrical surfaces and heads are numbered from 0, and the sectors are numbered from 1, so it can only represent 1024 cylinders× 63 sectors× 256 heads× 512byte=8455716864byte. i.e. the usual 8.4GB (which should actually be

7.8GB or so) limit. In fact, the number of heads is usually only 255 (determined by the assembly language's addressing registers), and even if you address these 3 bytes linearly, it is still not enough. In later operating systems, partitions over 8.4GB are no longer addressed by C/H/S. Instead, the total number of sectors occupied by the partition is expressed by offsetting CH~offset FH for a total of 4 bytes of 32-bit linear sector addresses. It can be seen that 4 bytes can represent 2³² sectors, that is, 2TB=2048GB, which is already an astronomical amount for most computers at present. On partitions that do not exceed 8.4GB, the partition size is the same as that of linear sectors. That is, the two methods of representation are coordinated. Even if it is not coordinated, linear addressing prevails. (Errors may be prompted in some systems). Partition end C/H/S over 8.4GB is generally populated as FEH FFH FFH. That is, the maximum value that can be expressed by C/H/S. Sometimes the column is filled with a 1024 die. But it doesn't really matter what these bytes are. Although today's systems use linear addressing to handle the size of partitions. But the principle of not crossing the cylinder remains unchanged. The total number of sectors in this partition plus the number of reserved sectors between the previous partition and the previous partition must still be an integer multiple of the cylindrical capacity. (The first sector in the reserved sector is the sector where the MBR or virtual MBR of the partition table is stored, and the total number of sectors in the partition is not counted in the reserved sector in linear expression.) If it is the first partition, the reserved sector is all sectors before this partition. Attachment: The partition table type flag is shown in Figure 4

分区类型标志:	
00 空, micosoft不允许使用。	63 GNU HURD or Sys
01 FAT32	64 Novell Netware
02 XENIX root	65 Novell Netware
03 XENIX usr	70 Disk Secure Mult
04 FAT16 <32M	75 PC/IX
05 Extended	80 Old Minix
06 FAT16	81 Minix/Old Linux
07 HPFS/NTFS	82 Linux swap
08 AIX	83 Linux
09 AIX bootable	84 OS/2 hidden C:
0A OS/2 Boot Manage	85 Linux extended
0B Win95 FAT32	86 NTFS volume set
0C Win95 FAT32	87 NTFS volume set
0E Win95 FAT16	93 Amoeba
0F Win95 Extended(>8GB)	94 Amoeba BBT
10 OPUS	A0 IBM Thinkpad hidden
11 Hidden FAT12	A5 BSD/386
12 Compaq diagnost	A6 Open BSD
16 HiddenFAT16	A7 NextSTEP
14 Hidden FAT16<32GB	B7 BSDI fs
17 Hidden HPFS/NTFS	B8 BSDI swap
18 AST Windows swap	BE Solaris boot
1B Hidden FAT32	partition
1C Hidden FAT32 partition	C0 DR-DOS/Novell DOS
(using LBA-mode	secured partition
INT 13 extensions)	C1 DRDOS/sec
1E Hidden LBA VFAT partition	C4 DRDOS/sec
24 NEC DOS	C6 DRDOS/sec
3C Partition Magic	C7 Syrinx
40 Venix 80286	DB CP/M/CTOS
41 PPC PreP Boot	E1 DOS access
42 SFS	E3 DOS R/O
4D QNX4. x	E4 SpeedStor
4E QNX4. x 2nd part	EB BeOS fs
4F QNX4. x 3rd part	F1 SpeedStor
50 Ontrack DM	F2 DOS 3.3+ secondary
51 Ontrack DM6 Aux	partition
52 CP/M	F4 SpeedStor
53 oNtRACK DM6 Aux	FE LAN step
54 OnTrack DM6	FF BBT
55 EZ-Drive	
56 Golden Bow	
5C Priam Edisk	
61 Speed Stor	

www.sjhf.net

图4

[Back to Index]

3.2 Extended Partition: Each logical drive in the Extended Partition has an Extended Boot Record (EBR) similar to MBR, which is also called virtual mbr or extended mbr, which means the same. The extended boot record includes an extended partition table and the labels for that sector. The extended boot record will record information that contains only the first face of the first cylinder of each logical drive in the extended partition. The boot sectors in a logical drive are typically located in relative sectors 32 or 63. However, if there is no extended partition on the disk, then there will be no extended boot record and logical drive. The first entry in the extended partition table of the first logical drive points to its own boot sector. The second term points to the EBR of the next logical drive. If no further logical drivers exist, the second term is not used and is recorded as a series of zeros. If there is an additional logical drive, then the first entry in the extended partition table of the second logical drive points to its own boot sector. The second entry of the extended partition table for the second logical drive points to the EBR of the next logical drive. The third and fourth items of the extended partition table are never used.

A 4-partition disk structure diagram shows the general organization of disks. As shown in Figure 5:

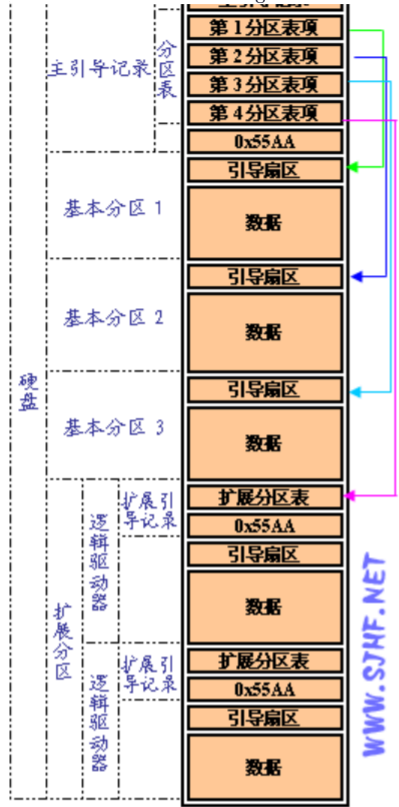


图5 一个4分区的基本磁盘

[Back to Index]

With regard to the extended partition, as shown in Figure 6, the extended boot record for the logical drives in the extended partition is a connection table. The diagram shows three logical drives on an extended partition, illustrating the difference between the previous logical drive and the last logical drive in the extended partition table.

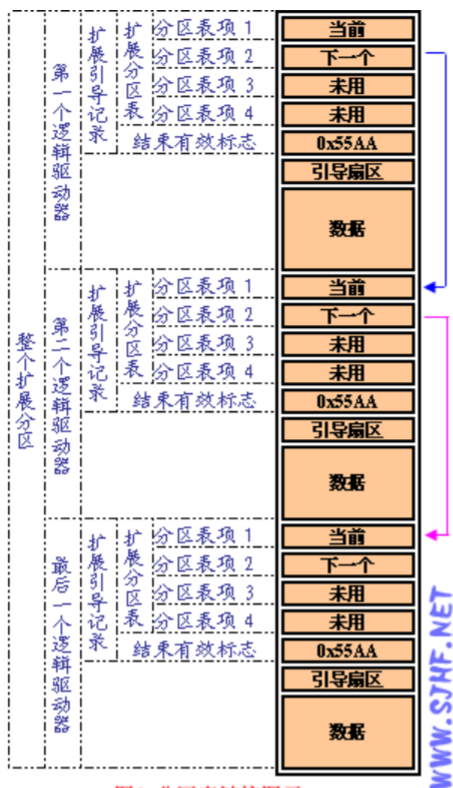


图6 分区表链接图示

[Back to Index]

With the exception of the last logical drive on the expansion partition, the format of the extended partition table described in Table 2 is repeated in each logical drive: the first item identifies the boot sector of the logical drive itself, and the second item identifies the EBR of the next logical drive. The extended partition table of the last logical drive will only list its own partition items. The second to fourth items of the last extended partition table are used.

Table 2 Expanding the contents of a partition table key

extended partition table entries	The contents of the partition table entry
The first item	Including the start address of the data with the current logical drive in the extended partition Information
The second item	Information about the next logical drive in the extended partition, including the address of the sector that contains the EBR of the next logical drive. If no further logical drives exist, the field will not be used use

The third item	Not used
The fourth item	Not used

[Back to Index]

The relative sector number field in the extended partition table key shows the number of bytes of the offset from the extended partition to the first sector in the logical drive. The number in the total sector number field refers to the number of sectors that make up the logical driver. The value of the total sector number field is equal to the number of sectors from the boot sector defined by the extended partition table key to the end of the logical drive.

Sometimes there is free space at the end of the disk, what is the remaining space? As we mentioned earlier, partitions are based on the capacity of 1 cylinder as the partition granularity, so if the total disk space is not an integer cylinder, the remaining space that is not enough for one cylinder is the remaining space. Logically speaking, the physical mode of the disk determines that the total capacity of the disk should be the capacity of an integer cylinder, so why is there not enough space for a cylinder? In my understanding, in order to make greater use of space, the current disk is generally not physically managed according to the sector of the outer sector is larger than the sector of the inner circle, but is only abstracted to be compatible with the operating system. Perhaps its actual space capacity may not be exactly the capacity of an integer cylindrical surface. Please let <http://www.sjhf.net> or zymail@vip.sina.com know if you have any insights.

[Back to Index] 4. The principle of FAT partitioning.

Let's start with a structural drawing:

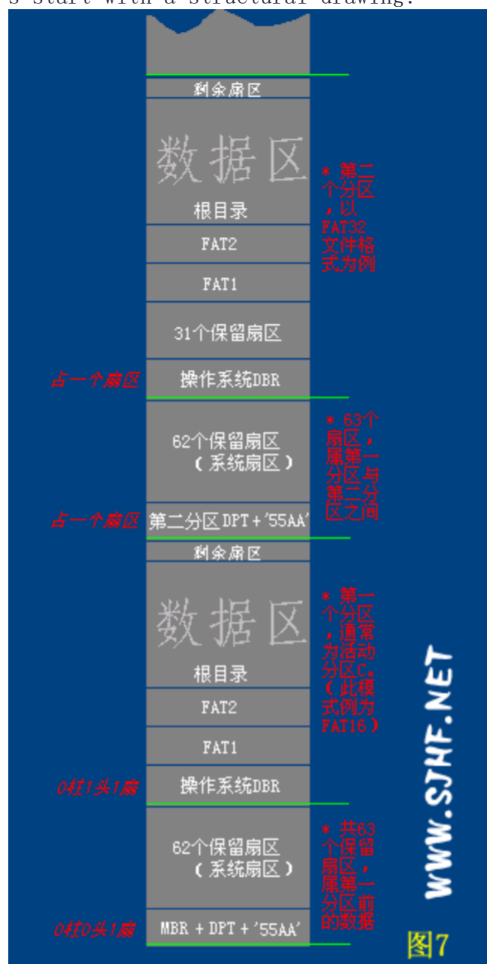


图7

[Back to Index]

Now we focus on how data is stored in FAT format partitions. The FAT partition format is the earliest partition format supported by MICROSOFT, and is divided into three format "variants" according to the number of bits occupied by each cluster chain in the FAT table (about the concept, which will be discussed later), but its basic storage methods are similar.

Carefully examine the composition of the FAT16 and FAT32 partitions in Figure 7. The following explains the concepts of DBR, FAT1, FAT2, root, data area, and remaining sector. The addresses mentioned are offset within the partition unless otherwise noted.

[Back to Index]

4.1 About DBR.

DBR area (DOS BOOT RECORD) means operating system boot record area, which usually occupies a total of 512 bytes in sector 0 of the partition (special cases also occupy other reserved sectors, let's talk about sector 0 first). In these 512 bytes, it is actually composed of jump instructions, vendor logos and operating system version numbers, BPB (BIOS Parameter Block), extension BPB, os bootloader, and end flag. Take FAT32, which is the most commonly used, as an example to illustrate the meaning of each byte of partitioned DBR. See Figure 8.

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	对应字符
00000000	EB	58	90	4D	53	57	49	4E	34	2E	31	00	02	08	20	00	隔态 SWIN4.1...
00000010	02	00	00	00	00	F8	00	00	3F	00	FF	00	3F	00	00	00	?.?. .?...
00000020	3F	04	7D	00	32	1F	00	00	00	00	00	00	02	00	00	00	?}.2.....
00000030	01	00	06	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	80	00	29	FE	1C	39	33	4E	4F	20	4E	41	4D	45	20	20	€.)?93NO NAME
00000050	20	20	46	41	54	33	32	20	20	20	FA	33	C9	8E	D1	BC	FAT32 ?庵鸭
00000060	F8	7B	8E	C1	BD	78	00	C5	76	00	1E	56	16	55	BF	22	鴉鵲經.狀..V.U?
00000070	05	89	7E	00	89	4E	02	B1	0B	FC	F3	A4	8E	D9	BD	00	.堦.堦.? 伴.
00000080	7C	C6	45	FE	0F	8B	46	18	88	45	F9	38	4E	40	7D	25	堦?堦.圖?N@}%
00000090	8B	C1	99	BB	00	07	E8	97	00	72	1A	83	EB	3A	66	A1	婆櫛..鉸.r.泮:f?
000000A0	1C	7C	66	3B	07	8A	57	FC	75	06	80	CA	02	88	56	02	. f;.奧驢.e?均.
000000B0	80	C3	10	73	ED	BF	02	00	83	7E	16	00	75	45	8B	46	e?s 颯..僂..uE 婢
000000C0	1C	8B	56	1E	B9	03	00	49	40	75	01	42	BB	00	7E	E8	.媯..?.I@u.B?~?
000000D0	5F	00	73	26	B0	F8	4F	74	1D	8B	46	32	33	D2	B9	03	_ .s&傍0t.婢23夜.
000000E0	00	3B	C8	77	1E	8B	76	0E	3B	CE	73	17	2B	F1	03	46	.;莖.燥.;蝶.+?F
000000F0	1C	13	56	1E	EB	D1	73	0B	EB	27	83	7E	2A	00	77	03	..V.胙s.?僂*.w.
00000100	E9	FD	02	BE	7E	7D	AC	98	03	F0	AC	84	C0	74	17	3C	堦.緯]瑯.颯勃t.<
00000110	FF	74	09	B4	0E	BB	07	00	CD	10	EB	EE	BE	81	7D	EB	t.???.肺縵]?
00000120	E5	BE	7F	7D	EB	E0	98	CD	16	5E	1F	66	8F	04	CD	19	寔 }豚穆.^f??
00000130	41	56	66	6A	00	52	50	06	53	6A	01	6A	10	8B	F4	60	AVfj.RP.Sj.j.嫵
00000140	80	7E	02	0E	75	04	B4	42	EB	1D	91	92	33	D2	F7	76	e~..u.簪?懷3吟v
00000150	18	91	F7	76	18	42	87	CA	F7	76	1A	8A	F2	8A	E8	C0	.PE v.B 轉鱗.嫵嫵?
00000160	CC	02	0A	CC	B8	01	02	8A	56	40	CD	13	61	8D	64	10	?..談..葵@?a 尙.
00000170	5E	72	0A	40	75	01	42	03	5E	0B	49	75	B4	C3	0D	0A	^r.@u.B.^.Iu 疵..
00000180	49	6E	76	61	6C	69	64	20	73	79	73	74	65	6D	20	64	Invalid system d
00000190	69	73	6B	0D	0A	44	69	73	6B	20	49	2F	4F	20	65	72	isk..Disk I/O er
000001A0	72	6F	72	0D	0A	52	65	70	6C	61	63	65	20	74	68	65	ror..Replace the
000001B0	20	64	69	73	6B	2C	20	61	6E	64	20	74	68	65	6E	20	disk, and then
000001C0	70	72	65	73	73	20	61	6E	79	20	6B	65	79	0D	0A	00	press any key...
000001D0	6B	65	79	0D	0A	00	00	00	49	4F	20	20	20	20	20	20	key....IO
000001E0	53	59	53	4D	53	44	4F	53	20	20	20	53	59	53	7E	01	SYSMSDOS SYS~.
000001F0	00	57	49	4E	42	4F	4F	54	20	53	59	53	00	00	55	AA	.WINBOOT SYS..U

图8

[Back to Index] The corresponding explanation of Figure 8 is shown in Table 3

Table 3 Division of positions in the DBR on the FAT32 partition

byte	bits shift	Field length	Field name	Corresp onding diagram 8. Col or
0x00		3 bytes	Jump command	
0x03		8 bytes	Manufacturer logo and OS version number	
0x0B		53 bytes	BPB	
0x40		26 bytes	Extended BPB	
0x5A		420 bytes	bootstrap code	
0x01FE		2 bytes	Effective sign	

[Back to Index] Figure 9 gives winhex's explanation of the relevant parameters of Figure 8 DBR:

Boot Sector FAT32, 基础偏移量: 0		
Offset	标题	数值
0	JMP instruction	EB 58 90
3	OEM	MSWIN4.1
BIOS Parameter Block		
B	Bytes per sector	512
D	Sectors per cluster	8
E	Reserved sectors	32
10	Number of FATs	2
11	Root entries (unused)	0
13	Sectors (on small volumes)	0
15	Media descriptor (hex)	F8
16	Sectors per FAT (small vol.)	0
18	Sectors per track	63
1A	Heads	255
1C	Hidden sectors	63
20	Sectors (on large volumes)	8193087
FAT32 Section		
24	Sectors per FAT	7986
28	Flags	0
2A	Version	0
2C	Root dir 1st cluster	2
30	FSInfo sector	1
32	Backup boot sector	6
34	(Reserved)	00 00 00 00 00 00 00 00 00 00 00 00
40	BIOS drive (hex, HD=8x)	80
41	(Unused)	0
42	Ext. boot signature (29h)	29
43	Volume serial number (decimal)	859380990
43	Volume serial number (hex)	FE 1C 39 33
47	Volume label	NO NAME
52	File system	FAT32
1FE	Signature (55 AA)	55 AA

[Back to Index]

Based on the above legend, let's discuss the parameter meaning of each byte of DBR. MBR transfers CPU execution to the boot sector, so the first three bytes of the boot sector must be legitimate executable x86-based CPU instructions. This is usually a jump instruction that is responsible for skipping the next few unexecutable bytes (BPB and extended BPB) to the OS boot code section.

The jump instruction is followed by the 8-byte long OEM ID, which is a string that identifies the name and version number of the operating system that formatted the partition. To preserve compatibility with MS-DOS, typically Windows 2000 formats the disk with "MSDOS" recorded in this field on FAT16 and FAT32 disks

5.0", on NTFS disks (about ntfs, see separately), Windows 2000 records "NTFS".

Usually "MSWIN4.0" appears in the OEM ID field on disks formatted in Windows 95, and "MSWIN4.1" in the OEM ID field on disks formatted in Windows 95 OSR2 and Windows 98. The next paragraph, starting with the offset 0x0B, is a description of the information that enables the executable boot code to find the relevant parameters. Commonly referred to as BPB (BIOS Parameter Block), BPB generally starts with the same amount of displacement, so the standard parameters are all in a known position. Disk capacity and geometry variables are enclosed in BPB. Since the first part of the boot sector is an x86 jump instruction. Therefore, in the future, the BPB can be extended by attaching new information to the end of the BPB. Only a small adjustment to the jump instruction is needed to adapt to the change in BPB. Figure 9 has listed the names and values of the items, and for the sake of systematic research, the meaning of BPB and the meaning of extended BPB in FAT32 partition format are interpreted in tables for Figure 8, see Tables 4 and 5.

Table 4 BPB fields for FAT32 partitions

Byte displacement	Field length (bytes)	Figure 8 corresponds to value	Names and definitions
			Bytes PerSector The size of the hardware sector. root
0x0B	2	0x0200	The decimal values of the field are 512, 1024, 2048, and 4096. For most disks, the value of this field is 512
			Sectors PerCluster, the number of sectors in a cluster. Since the FAT32 file system can only be traced
			There are a limited number of clusters (up to 4 294 967296), so by increasing the number of sectors per cluster, the FAT32 file system can support the maximum number of partitions. The default cluster size for a partition depends on the size of that partition. The legal decimal of this field
0x0D	1	0x08	The values are 1, 2, 4, 8, 16, 32, 64, and 128. Windows 2000 FAT32 implementations can only create partitions up to 32GB. However, Windows 2000 is able to access the services provided by other operating systems (Windows 95, OSR2 and its operating systems later versions).
			district
			ReservedSector The number of sectors before the first FAT starts, including boot sectors. root
0x0e	2	0x0020	The decimal value of the field is generally 32
			Number of FAT The number of copies of FAT on the partition. of this field
0x10	1	0x02	The value is generally 2
0x11	2	0x0000	根目录项数(Root Entries)只

			There are FAT12/FAT16 that use this field. For FAT32 partitions, this field is required Must be set to 0
0x13	2	0x0000	Small Sector (only FAT12/FAT16 uses this field) This field is required for FAT32 partitions Must be set to 0
0x15	1	0xF8	Media Descriptors provide information about the media being used. A value of 0xF8 indicates hard Disk, 0xF0 indicates a high-density 3.5-inch floppy disk. Media descriptors are to be used for MS-DOS FAT16 disk, which is not used in Windows 2000
0x16	2	0x0000	The number of sectors per FAT (Sectors PerFAT) is only used by FAT12/FAT16, which is the original word for FAT32 partitions The segment must be set to 0
0x18	2	0x003F	Sectors PerTrack contains the Sectors PerTrack geometry value for disks using INT13h. This partition is surrounded by a cylindrical surface of multiple heads It is divided into multiple magnetic tracks
0x1A	2	0x00FF	Number of Head This field contains the Number of Heads geometry value for disks using INT 13h. example For example, in a 1.44MB 3.5-inch card The value of this field is 2
0x1C	4	0x0000003F	Hidden Sector The number of sectors on the partition before the boot sector. This value is used in the process of calculating the absolute displacement of the data region of the boot sequence to the root directory. This field is generally only meaningful for media that are visible on the interrupt 13h. On media without partitions It must always be 0
0x20	4	0x007D043F	Large Sector This field contains the total number of sectors in the FAT32 partition District number
0x24	4	0x00001F32	Sectors PerFAT (used by FAT32 only) The number of sectors per FAT in that partition. compute The machine uses this number and the number of FAT, as well as the number of hidden sectors (described in this table), to determine where to start the root directory. The computer can also determine from the number of items in the directory where the user data area of that partition comes from begins
0x28	2	0x00	Extended Flag (used only by FAT32) for the two byte nodes

The values for each of the components are: Bits 0-3: Number of active FAT (counting from 0, not 1).

Bits 4-6 are valid only when not mirroring: The reserved bit 7:0 value means that FAT is mapped to all FATs at runtime

A value of 1 indicates that only one FAT is active in bits 8-15: Retained

The file system version (File systemVersion) is only for FAT32, with high bytes being the primary revision number and low bytes being the minor revision number. This field supports future extensions to this FAT32 media type. If this field is non-zero, it will not be supported in previous versions of Windows

0x2A 2 0x0000

Partitions

Root ClusterNumber (for FAT32 only) The cluster number of the first cluster in the root directory. of this field

0x2C 4 0x00000002

The value is usually 2, but not always

File system information sector number (File System InformationSectorNumber)

(只供 FAT32 使

with) in the reservation area of the FAT32 partition File System Information (FSINFO) structure

0x30 2 0x0001

sector code. Its value is generally 1. In preparation A copy of the FSINFO structure is kept in the Backup BootSector, but this copy is maintained

Not kept updated

The backup boot sector (for FAT32 only) is a non-zero value that represents the sector number in the reserved area where the partition holds a copy of the boot sector. root

0x34 2 0x0006

The field typically has a value of 6, which is not recommended Use other values

Reserved (for FAT32 only) Reserved space for future expansion. This field is a total of 0

0x36 12 All 12 bytes are 0x00

[Back to Index]

Table 5 Extended BPB field for FAT32 partitions

Byte	Field	Figure 8 corresponds	Field name and definition
displa	lengt	value	
cement h			

		(bytes)	
			The physical drive number is the same as the BIOS physical drive letter related. Floppy disk drives are identified as 0x00, physical hard drives are identified as 0x80, and with physical disks
0x40	1	0x80	Drive agnostic. In general, set this value before making an INT13h BIOS call, specifying the device being accessed. This value is only available if the device is a boot device
			make sense
0x41	1	0x00	保留(Reserved) FAT32 分区 Always set the value of this field to 0
			ExtendedBoot Signature This field must be compatible with Windows 2000
0x42	1	0x29	The value of the identification is 0x28 or 0x29
			Volume SerialNumber A random serial number generated when formatting a disk
0x43	4	0x33391CFE	Helps to differentiate between disks
			Volume Label This field It can only be used once, and it is used to save the volume label number. Now the label is saved in the root as a special file
0x47	11	"NO NAME"	In the record
0x52	8	"FAT32"	System ID: FAT32 is generally used as "FAT32" in the file system.

[Back to Index]

The offset of the DBR 0x5A the data that starts is the operating system boot code. This is pointed to by the jump command that starts with offset 0x00. The jump instruction "EB 58 90" with an offset of 0x00~0x02 listed in Figure 8 clearly indicates the offset position of the OS boot code. jump 58H plus the amount of displacement required for the jump command, which starts at 0x5A. The content of this instruction is also different on different operating systems and different boot methods. Most of the information says win98, built on win2000 on the basic FAT partition, and the DBR used by winxp only occupies sector 0 of the basic partition. They mentioned that for FAT32, the general 32 basic partitions reserve sectors with only sector 0 are useful. In fact, if the operating system built with FAT32 is win98, the system will use sectors 0 and 2 of the basic partition to store the OS boot code; Operating systems built with FAT32 If they are win2000 or winxp, the system stores the OS boot code using sectors 0 and 0xC of the base partition (win2000 or winxp, whose 0xC position is indicated by the 0xAB offset of sector 0). Therefore, in the FAT32 partition format, if the contents of a sector of DBR are correct and the second sector (win98 system) or sector 0xC (win2000 or winxp system) is missing, the system will not be able to boot. If you manually set up the NTLDR dual system yourself, you must know this.

The last two bytes of the DBR sector generally store the value of 0x55AA of the DBR valid flag, for the others

The system will not execute DBR-related instructions. Several other sectors involved in OS boot mentioned above are also required to be marked with 0x55AA as a legal end mark.

[返回索引]FAT16 DBR:

The meaning of DBR in FAT32 is roughly like this, for FAT12 and FAT16 its basic meaning is similar, but there is a small difference in the meaning of the relevant offset and parameters, the difference and origin of the FAT format will be discussed later, and FAT12 and FAT16 will not be mentioned here. I will list the sector parameter meanings of FAT16. Interested friends should research it themselves, it is similar to FAT32.

Table 6 Boot sector byte bits on a FAT16 partition

Field name	Field number	Field length (bytes)
Jump command (Jump Instruction)	0x00	3
OEM ID	0x03	8
BPB	0x0B	25
Extended BPB	0x24	26
Bootstrap code (Code)	0x3E	448
End of sector identifier (0x55AA)	0x01FE	4

[Back to Index]

Table 7 BPB fields for FAT16 partitions

Field name	Field number	Field length (bytes)	Example	Names and definitions
Bytes PerSector	0x0B	2	0x0200	The size of the hardware sector. The legitimate decimal values in this field are: 512, 1024, 2048, and 4096. For most disks, the value of this field is 512
Sectors PerCluster	0x0D	1	0x40	The number of sectors in a cluster. Since the FAT16 file system is only 2GB, it can track a limited number of clusters (up to 65,536). Therefore, by increasing the number of sectors per cluster can support the maximum number of partitions. The default cluster of the partition

			The size depends on the size of the partition. The legitimate decimal value for this field There are 1, 2, 4, 8, 16, 32, 64 and 128. Resulting in clusters larger than 32KB (bytes per sector * per cluster fan number of zones) will cause disk errors and software bugs ReservedSector The first FAT starts
0x0e	2	0x0001	The number of sectors in front, including the guide sector. The decimal values for this field are average for 1
0x10	1	0x02	FAT 数(Number of FAT)该分 The number of copies of FAT on the zone. This field is generally 2 The number of root entries can be saved in the root directory of the partition 32 bytes long in the folder The total number of file and folder name items. On a typical hard drive, The value for this field is 512. One of them
0x11	2	0x0200	Individual items are often used as volume labels, and files and folders with long names use multiple items per file. of the file and folder items The maximum number is usually 511, but if long file names are used, they are often all This number cannot be reached Small Sector The number of sectors on the partition, expressed as: 16-bit (<65536). For partitions larger than 65,536 sectors, the value of this field is 0, and large is used sector to replace it
0x13	2	0x0000	Media Descriptors provide information about the media being used. A value of 0xF8 indicates a hard drive and 0xF0 indicates a high density 3.5" floppy disk. Media descriptors are intended for use on MS-DOS FAT16 disks and are not used in Windows 2000 use
0x15	1	0xF8	每 FAT 扇区数(Sectors for FAT) The number of sectors occupied by each FAT on that partition. The computer uses this number and the number of FAT and the number of hidden sectors to decide where the root directory starts. The computer can also decide where the user data area of that partition starts based on the number of items in the root directory (512).
0x16	2	0x00FC	

0x18	2	0x003F	每道扇区数(Sectors for Trark)
0x1A	2	0x0040	磁头数(Number of head)
0x1C	4	0x0000003F	Hidden Sector The number of sectors on the partition before the boot sector. This value is used in the process of calculating the absolute displacement of the boot sequence to the root directory and data area
0x20	4	0x003EF001	Large Sector If the value of the small sector number field is 0, this field contains the total number of sectors in the FAT16 partition. If small The value of the sector number field is not 0, that The value of this field is 0

[Back to Index]

Table 8 Extended BPB field for FAT16 partitions

Byte displa cement	length (byt es)	Field length Figure 8 corresponds value	Field name and definition
			The PhysicalDrive Number is related to the physical drive letter in the BIOS. floppy disk drive device is identified as 0x00, physical hard disk is identified as 0x80, and with objects
0x24	1	0x80	It has nothing to do with disk drives. So so , set this value before making an INT13hBIOS call, specifying the device being accessed. Only if the device is a boot device This value is meaningful
0x25	1	0x00	The Reserved FAT16 partition generally sets the value of this field to 0
0x26	1	0x29	ExtendedBoot Signature This field is required You need to be able to use Windows 2000 The value identified 0x28 or 0x29
0x27	2	0x52368BA8	Volume SerialNumber A random serial number generated when formatting a disk, which has it Helps to differentiate between disks
0x2B	11	"NO NAME"	Volume Label This field can only be used once, and it is used to save the volume label number. Now, the roll label is been

Saved as a special file at the root
in the table of contents

File SystemType According
to the disk format, the

0x36 8 "FAT16" fields can have values of FAT,
FAT12 或 FAT16

[Back to Index]

4.2 About retaining sectors

At the offset 0x0E of the FAT file system DBR described above, the number of sectors reserved is stored in 2 bytes. The so-called reserved sectors (sometimes called system sectors, hidden sectors) refer to the sectors owned by the system starting from the partitioned DBR sectors, including DBR sectors. In the FAT16 file system, the data for reserved sectors is usually set to 1, which is just the DBR sector. In FAT32, the data of the reserved sectors is usually set to 32, sometimes the FAT32 partition divided with Partition Magic will set 36 reserved sectors, and some tools may set 63 reserved sectors.

The reserved sectors in FAT32 are not involved in operating system management and disk data management, except for the total sector 0 of the disk used as DBR, and the total sector 2 (win98 system) or total sector 0xC (win2000, winxp) as the OS boot code extension part. The reason why the operating system sets reserved sectors in FAT32 is to back up the DBR or to save it for future upgrades. In FAT32, the DBR offset 0x34 2 bytes of data indicates the location of the DBR backup sector, which is usually 0x06, or sector 6. When the FAT32 partition DBR sector is corrupted, making the partition inaccessible. You can replace sector 0 with the original backup of sector 6 to retrieve the data.

[Back to Index]

4.3 Storage principles of FAT tables and data.

FAT Table (File Allocation Table) is a chain structure introduced by Microsoft in the FAT file system for indexing and locating disk data (files). If we compare a disk to a book, the FAT table can be considered the equivalent of a table of contents in a book, and the file is the content of each chapter. However, the FAT table is represented in a very different way than a directory.

In the FAT file system, files are stored according to the cluster chain data structure defined by the FAT table. At the same time, the FAT file system also abstracts the directories used to organize data into files to simplify data management.

[Back to Index]

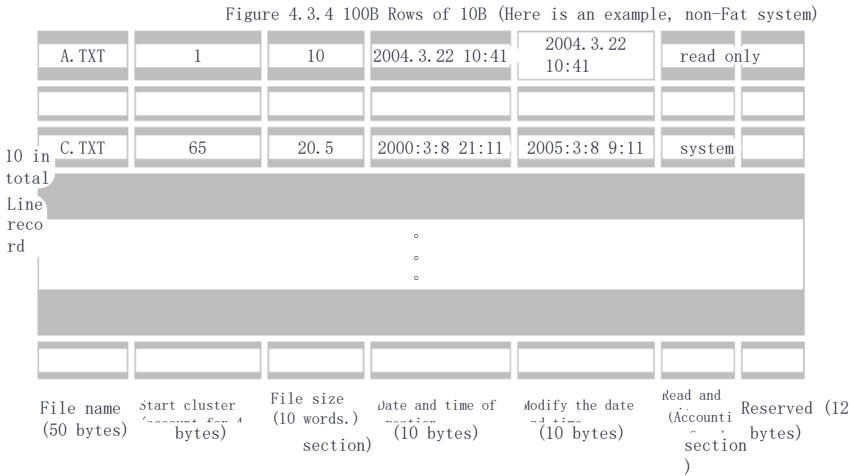
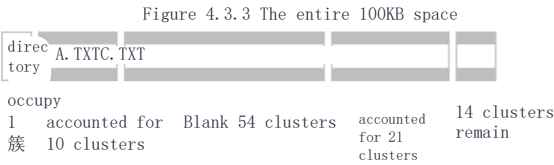
★ Stored procedure scenario:

We simulate the process of storing data in a partition to illustrate the principles of data storage in a FAT file system.

Suppose there is now an empty disk with no data at all, 100KB in size, and we imagine it as a linear spatial address. For the convenience of storage management, we artificially divide the 100KB space into 100 copies, 1KB each. Let's store these files in order: A.TXT (size 10KB), B.TXT (size 53.6KB), and C.TXT (size 20.5KB).

At the very least, we can think of these 3 files in this 100KB space in order. At the same time, don't forget that we also have to note their size and where they started so that the next time we need them, we will be able to find them

Sometimes it can't be solved.
 Then let's change the operation, delete B.txt, and free up the space of b.txt.
 At this time, the space is shown in Figure 4.3.3, and the table of contents is shown in Figure 4.3.4



This operation seems to be okay, so let's do it next, after saving a D.txt file (size is 60.3KB), a total of 100 clusters of space only 31 clusters are used, and 68 clusters are left, which can be put down. But? Where to put it? Without 61 contiguous spaces, directory lines cannot be written, and it seems that no contiguous block storage will work for the time being.

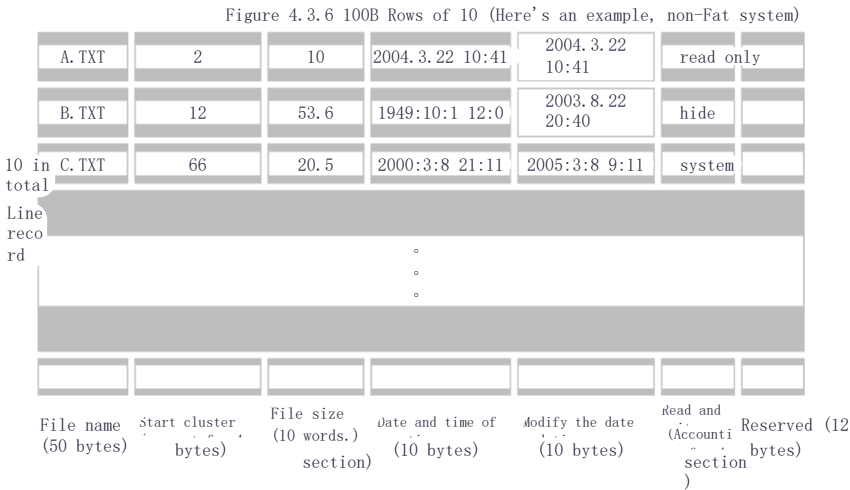
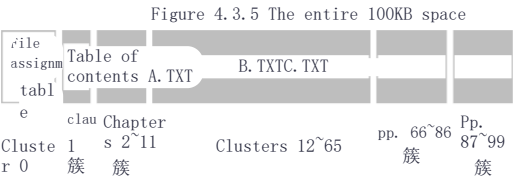
You can think of moving other files that affect our operation when there is not enough continuous space or increasing the length of the file to make space, but I have to ask you, is that not doing anything all day, or messing things upside down?

It seems that the file system we designed has a fatal vulnerability, how to fix it?

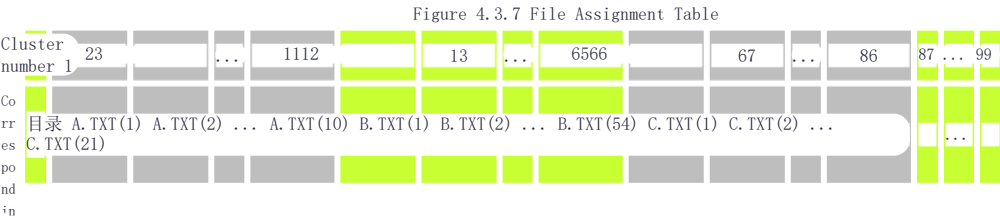
In fact, it can be solved like this: First, we allow discontinuous storage of files. The directory still only records the size of the starting cluster and the file. So how do we record which clusters are occupied by files, it is not convenient to map clusters with files, because the file names are not fixed. Let's think differently, we can use clusters to map files, leaving a few clusters in the front of the entire storage space to record the relationship between data and cluster numbers in the data area. For the above example, because the total space is not large, so

to record this correspondence with the space of 1Kb in the front, assuming that all 3 files are stored, the space allocation is shown in the figure

4.3.5, and modify the table of contents at the same time, as shown in Figure 4.3.6



The first cluster is used to record the occupancy of each cluster in the data area, and is temporarily called the file allocation table. Combined with the file allocation table and file directory, complete file reading can be achieved. We thought of making a file allocation table into a data table that records the correspondence between clusters and data in the form of Figure 4.3.7.



The organization in Figure 4.3.7 is completely possible to record the file ownership cluster. But it's not efficient enough. For example, the file name is recorded too much in the file allocation table, which is a waste of space, when in fact the beginning cluster of files is already recorded in the directory. Therefore, it can be improved to store the relationship of the possession cluster in a chain way, which becomes the organization method shown in Figure 4.3.8.

Figure 4.3.8 Improved File Allocation Table

Cluster 1 2 3 ... 11 12 13 ... 65 66 67 ... 86 87
... 99 Corresponding Data Catalog 3 4 ... FF 13 14
... FF 67 68 ... FF 00 ... 00

Refer to Figure 4.3.8 to understand the meaning of the file allocation table. For example, in the file a.txt, we can determine that the next cluster is 3 if the first cluster of a.txt specified in the directory item is 2, and then find the second cluster record of the file allocation table, and the 3 is registered on it, and we can determine that the next cluster is 3. If we find the 3rd cluster record of the file allocation table, and register 4 on it, we can determine that the next cluster is 4 until we point to the 11.....th cluster and find that the next cluster is FF, which is the end. The document was read without any mistakes.

Let's look at the third case mentioned above, which is to delete b.txt and save it into a 60.3KB d.txt. Utilizing cluster chains is easy to implement. The implemented disk is shown in the figure 4.3.9 4.3.10 4.3.11

Figure 4.3.9 The entire 100KB space division

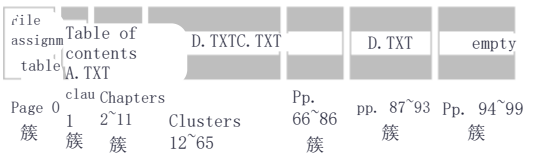


Figure 4.3.10 File Assignment Table

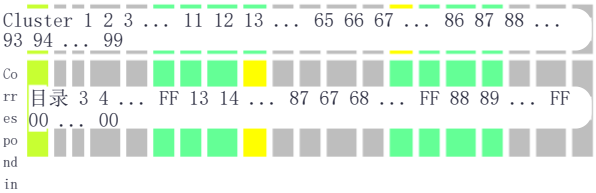
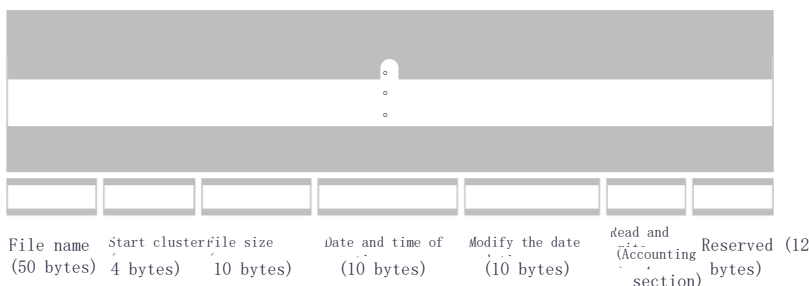


Figure 4.3.11 Table of Contents 100B Rows of 10 (Here is an example, non-Fat system)

10 in total	A.TXT	2	10	2004.3.22 10:41	2004.3.22 10:41	read only
Line reco rd	C.TXT	66	20.5	2000:3:8 21:11	2005:3:8 9:11	system
	D.TXT	12	60.3	1999:5:1 8:00	2003:3:20 14:0	archive



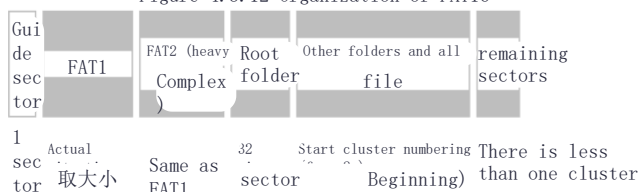
The above is one of our assumptions about file storage, and it's time to get rid of it. The above idea is actually the essence of the idea of the FAT file system (but no, especially since the meaning of specific parameters is completely different from the example we gave). Please forget the details above and try to remember the bottom).

[Back to index] ★FAT16
storage principle:

When a part of the disk space is formatted into a FAT file system, the FAT file system plans this partition as a whole allocable area to facilitate data storage. Generally speaking, the division form is shown in Figure 7. Let's extract the FAT16 part and describe it in detail:

FAT16 is an earlier file system launched by Microsoft, with high compatibility, and is still widely used in personal computers, especially mobile storage devices, FAT16 is simply composed of 6 parts (mainly the first 5 parts) shown in Figure 4.3.12. Boot Sectors (DBR) As we have already said, FAT16 does not leave any reserved sectors after the DBR, followed by the FAT table. The FAT table is used by FAT16 to record the cluster chain structure of disk data areas. Like the example we mentioned earlier, FAT divides disk space into a certain number of sectors, which are called clusters. Normally, the principle of 512 bytes per sector is unchanged. The size of a cluster is generally the size of 2n (n is an integer) sectors, such as 512B, 1K, 2K, 4K, 8K, 16K, 32K, 64K. It usually does not exceed 32K in practice. The reason why disks are allocated in clusters rather than sectors is because when the partition capacity is large, the use of sector management with a size of 512b will increase the number of items in the FAT table, increase the consumption of large file access, and the file system efficiency is not high. The size of the partition is related to the value of the cluster, see Table 9

Figure 4.3.12 Organization of FAT16



[Back to Index]

Table 9 FAT16 partition size and partition
space size for each cluster is large

		small
0MB-32MB	1	512 words section
32MB-64MB	2	1k
65MB-128MB	4	2k
129MB-225MB	8	4k
256MB-511MB	168k	
512MB-1023MB	32	16k
1024MB-2047MB	64	32k
2048MB-4095MB	12864k	

[Back to Index]

Note: For partitions with less than 32,680 sectors, the cluster space size can be up to 8 sectors per cluster.

Regardless of whether the user uses the disk manager to format the partition or uses the command prompt to type the format command, the formatter creates a 12-bit FAT. Partitions of less than 16MB, the system typically formats them as 12-bit FAT, FAT12 is the initial implementation of FAT, which is intended for small media. The FAT12 file allocation table is smaller than the FAT16 and FAT32 file allocation tables because it uses less space for each entry. This leaves more room for data. All formatted in FAT12

The 5.25-inch floppy disk and the 1.44MB 3.5-inch floppy disk are formatted in FAT12. Except that the number of binary bits recorded in the FAT table for each cluster of links is different from FAT16, the rest of the principles are the same as FAT16 and are no longer interpreted separately...

When formatting a FAT16 partition, the formatter determines the size of the cluster based on the size of the partition, and then determines the number of sectors required for the FAT table based on the number of sectors retained, the number of sectors in the root directory, the number of clusters that can be divided into data regions, and the space occupied by the FAT table itself, and then writes the calculated results to the relevant location in the DBR.

The offset 0x11 of the FAT16 DBR parameter records the number of sectors occupied by the root directory. The offset 0x16 records data for the sectors occupied by the FAT table. Offset 0x10 records the number of copies of the FAT table. After obtaining these parameters, the system can determine the starting sector offset of the data area.

The FAT16 file system processes data in clusters from the first sector after the 32 sectors occupied by the root directory, which was previously still in sectors. For the first cluster after the root directory, the system is not numbered as cluster 0 or cluster 1 (perhaps for the reason of the keyword), but as cluster 2, which means that the first cluster in the order of the data area is also the second cluster in the number.

The reason why there are 12, 16, and 32 different versions of the FAT file system is that the FAT table is used to record the number of binary bits of any cluster of links. In the case of FAT16, each cluster occupies 2 bytes (16 bits in binary) in the FAT table. Therefore, the maximum cluster number that FAT16 can represent is 0xFFFF (65535 in decimal), and the maximum disk space that FAT32 can manage with a cluster size of 32K is: $32KB \times 65535 = 2048MB$, which is why FAT16 does not support partitions exceeding 2GB.

The FAT table is actually a data table, measured in 2 bytes, let's call this unit FAT record items, usually its 1st and 2nd record items (the first 4 bytes) are used as media descriptions. Start from the third record item to record the cluster chain of files and folders other than the root directory. The performance of the cluster FAT is described by the corresponding value, see Table 10

Table 10 FAT16 Values for record items
Meaning (hexadecimal)

The value of the FAT16 record item	The performance of the corresponding cluster
0000	Unassigned 簇
0002~FFEF	assigned 簇
FFF0~FFF6	系统保留
FFF7	坏簇
FFF8~FFFF	End of file 簇

Look at a file allocation table for FAT16 captured by winhex, Figure 10:

[Back to Index]

驱动器 L:																	访问
Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00000020	F8	FF	FF	FF	FF	2FF	05	800	06	400	FF	5FF	07	600	08	700	?
00000028	09	800	0A	900	0B	100	0C	100	0D	200	0E	100	FF	14FF	10	00	
000000220	11	00	12	00	13	00	14	00	15	00	16	00	17	00	18	00	
000000230	19	00	1A	00	1B	00	1C	00	1D	00	1E	00	1F	00	20	00	
000000240	21	00	22	00	23	00	24	00	25	00	26	00	27	00	28	00	!. ". #. \$. %. &. ' . (.
000000250	29	00	2A	00	2B	00	2C	00	2D	00	2E	00	2F	00	30	00	). *. +. , . - . / . 0 .
000000260	31	00	32	00	33	00	34	00	35	00	36	00	37	00	38	00	1. 2. 3. 4. 5. 6. 7. 8. .
000000270	39	00	3A	00	3B	00	3C	00	3D	00	3E	00	3F	00	40	00	9. :. ;. <. =. >. ? . @ .
000000280	41	00	42	00	43	00	44	00	45	00	46	00	47	00	48	00	A. B. C. D. E. F. G. H. .
000000290	49	00	4A	00	4B	00	4C	00	4D	00	4E	00	4F	00	50	00	I. J. K. L. M. N. O. P. .
0000002A0	51	00	52	00	53	00	54	00	55	00	56	00	57	00	58	00	Q. R. S. T. U. V. W. X. .
0000002B0	59	00	5A	00	5B	00	5C	00	5D	00	5E	00	5F	00	60	00	Y. Z. [. \.]. ^. _ . `.
0000002C0	61	00	62	00	63	00	64	00	65	00	66	00	67	00	68	00	a. b. c. d. e. f. g. h. .
0000002D0	69	00	6A	00	6B	00	6C	00	6D	00	6E	00	6F	00	70	00	i. j. k. l. m. n. o. p. .
0000002E0	71	00	72	00	73	00	74	00	75	00	76	00	77	00	78	00	q. r. s. t. u. v. w. x. .
0000002F0	79	00	7A	00	7B	00	7C	00	7D	00	7E	00	7F	00	80	00	y. z. {. . }. ~. . €.
000000300	81	00	82	00	83	00	84	00	85	00	86	00	87	00	88	00	????????}. ~. . €.
000000310	89	00	8A	00	8B	00	8C	00	8D	00	8E	00	8F	00	90	00	????????}. ~. . €.
000000320	91	00	92	00	93	00	94	00	95	00	96	00	97	00	98	00	????????}. ~. . €.
000000330	FF	FF	9A	00	FF	FF	9C	00	9D	00	FF	FF	9F	00	A0	00	? ?? ?
000000340	FF	FF	FF	FF	FF	FF	A4	00	A5	00	A6	00	A7	00	A8	00	?????
000000350	A9	00	AA	00	AB	00	AC	00	AD	00	AE	00	AF	00	B0	00	????????
000000360	B1	00	B2	00	B3	00	B4	00	B5	00	B6	00	B7	00	B8	00	????????
000000370	B9	00	BA	00	BB	00	BC	00	BD	00	BE	00	BF	00	C0	00	????????
000000380	C1	00	C2	00	C3	00	C4	00	C5	00	C6	00	C7	00	C8	00	????????

[Back to Index]

As shown in the figure, the FAT table starts with "F8 FF FF FF", which is a media description unit and does not participate in the FAT table cluster chain relationship. The small red letters indicate the cluster number corresponding to every 2 bytes of the FAT

Relative offset 0x4~0x5 The offset is the 2nd cluster (1st cluster in order), here it is FF, which means that the file (directory) stored on the 2nd cluster is a small file, and only 1 cluster is over.

The data stored in Cluster 3 is 0x0005, which is the first cluster of a file or folder. Its content is cluster 5, which means that the next cluster is located in cluster 5 - the FAT table guides us to the point of cluster 5 of the FAT table, and the data written on it is "FF FF", which means that this file has reached the tail cluster.

The data stored in Cluster 4 is 0x0006, which is again the first cluster of a file or folder. Its content is cluster 6, that is, the next cluster is located in cluster 6 - the FAT table guides us to the 6th cluster of the FAT table, and the data written on it is 0x0007, that is, the next cluster is located in the 7th cluster - the FAT table leads us to the 7th cluster of the FAT table point..... Until the relative offset of the sector is read according to the FAT chain, 0x1A~0x1B, that is, the 13th cluster, the data written on it is 0x000E, that is, it points to the 14th cluster - the content of the 14th cluster is "FF FF", which means that this file has reached the tail cluster.

The following FAT table data is the same as above. No more analysis.

The FAT table records the stored linked list of disk data files and is extremely important for the reading of the data, so much so that Microsoft has created a backup of the FAT table in the FAT file system it has developed, which is what we see FAT2. The content of FAT2 and FAT1 is usually synchronized instantly, that is, if it passes positive

If FAT1 is changed by normal system read/write, then FAT2 will also be updated. If you look at it from this point of view, this function of the system is a natural disaster when data is recovered.

The directory structure of the FAT file system is actually a directed tree from root to leaf, and the directed refers to the fact that any file (including folders) in the FAT partition needs to be addressed from the root directory. Think of it this way: the entrance to the directory storage structure is the root directory.

The FAT file system addresses other files (including folders) based on the root directory, so the location of the root directory must be determined before the disk can access the data. The FAT file system is determined based on the relevant DBR parameters of the partition and the size of the calculated FAT tables (2 copies) stored in the DBR. After formatting, the size and location of the directory are already determined: the location is immediately after FAT2, which is usually 32 sectors in size. After the root directory is the data area cluster 2.

An important idea of the FAT file system is to treat directories (folders) as special files, and FAT32 even treats the root directory as a file (side: NTFS abstracts many things such as partition parameters, security permissions, etc. into files is a sublimation of this idea), in FAT16, although the root directory status is not equivalent to an ordinary file or directory, its organizational form is no different from that of an ordinary directory (folder). All folder (directory) files in a FAT partition can actually be regarded as a data table that stores the entry parameters of other files (folders). So the size of the directory's footprint is not equal to the size of all the data under it, but it is not the same as 0. It usually takes up a small space and can be regarded as a directory file as a simple two-dimensional table file. Its specific storage principle is:

It doesn't matter how many clusters of directory files take up space, how many bytes a cluster is. The system allocates the clusters occupied by directory files in 32-byte increments. These 32 bytes define the properties of a file (or folder) in this directory with a definite offset, which is actually a simple two-dimensional table.

The definitions of each byte offset for these 32 bytes are shown in Table 11:

Table 11 FAT16 is a 32-byte representation definition for a directory item

Byte Offset (16)	Number of bytes	definition
0x0~0x7	8	filename
0x8~0xA	3	Extension: 00000000 (Read and Write)
		00000001 (Read Only)
		00000010 (hidden)
0xB	1	attribute bytes 00000100 (System)
		00001000 (Label)
		00010000 (subdirectory)
		00100000 (Archive)
0xC~0x15	10	System retention
0x16~0x17	2	The most recent modification of the file
0x18~0x19	2	The date of the most recent modification of the document
0x1A~0x1B	2	Indicates the first cluster number of the file
0x1C~0x1F	4	Indicates the length of the file

[\[Back to Index\]](#)

Some values in Table 11 are explained: (1) For short file names, the system divides the file name into two parts for storage, that is, the main file name + extension. 0x0~0x7 byte The main file name of the record file, 0x8~0xA the extension of the record file, and take the ASCII code value in the file name. Do not record the "." between the main file name and the extension. If the main file name is less than 8 characters, it is filled with a space character (20H), and if the extension is less than 3 characters, it is also filled with a space character (20H). 0x0 If the value of the offset is 00H, it means that the catalog item is empty; If it is E5H, it indicates that the directory item has been used, but the corresponding file or folder has been deleted. (This is also the theoretical basis for reverting after accidental deletion). If the first character in the file name is "." or ".." This cluster records a directory item for a subdirectory. "." Represents the current directory; ".." Represents the parent directory (the same meaning as our use in DOS or Windows, if the disk data is destroyed, you can infer the starting position of the data area of the disk through the specific parameters of these two directory items, guess the size of the cluster, etc., so it is more important)

(2) Attribute field of 0xB: It can be regarded as a system dividing a byte of 0xB into 8 bits, and one of them represents the presence or absence of a certain attribute. In this way, each of the 8 bits in a byte takes a different value to reflect the different values of each attribute. For example, 00000101 means that this is a file, and the attribute is read-only, system.

(3) 0xC~0x15 are reserved unused in the original FAT16 definition. In higher versions of WINDOWS systems, it is also sometimes used to record the time of modification and the most recent access. The meaning of the field is the same as the definition of FAT32, see FAT32 below.

(4), Time in 0x16~0x17 = hours * 2048 + minutes * 32 + seconds / 2. The result can be converted into hexadecimal filling. That is, the 0~4 bits of 0x16 bytes are measured in 2 seconds; 5~7 bits of 0x16 bytes and 0~2 bits of 0x17 bytes are minutes; 3~7 bits of 0x17 bytes are hours.

(5), the date in 0x18~0x19 = (year-1980) * 512 + month * 32 + day. The resulting result can be converted into hexadecimal filling. That is: 0x18 bytes 0~4 bits are the number of dates; 0x18 bytes 5~7 bits and 0x19 bytes 0 bits are months; The 1~7 digits of 0x19 bytes are the year number, and the original definition of 0~119 represents 1980~2099, and the current high version of Windows allows 0~127, that is, the year number can be up to 2107.

(6) 0x1A~0x1B the first cluster number of the file or directory that represents the file, the system finds the entry in the FAT table according to the first cluster number it holds, and then tracks the cluster chain to the end of the cluster, and at the same time uses bytes at 0x1C~0x1F bytes to determine the validity. You can read the file (directory) completely without error.

(7) The addressing process of ordinary subdirectories is also specified by the directory items in its parent directory, which is different from data files (referring to non-directory files) in that the directory item is offset 0xB position 4 1, while the data file is 0.

For the entire FAT partition, the cluster allocation is not always allocated cleanly. For example, in a FAT system with 99 sectors, if the cluster size is set to 2 sectors, there will be 1 sector that cannot be assigned to any cluster. This is the remaining sector of the partition, which is located at the end of the partition. Some systems use the last remaining sector to back up the DBR of this partition, which is also a good backup method.

The early FAT16 system did not have long file names, but the Windows operating system has fully supported long file names on FAT16. The definition of FAT16 long file names is the same as that of FAT32 long file names, and the long file names will be explained in detail in the FAT32 section.

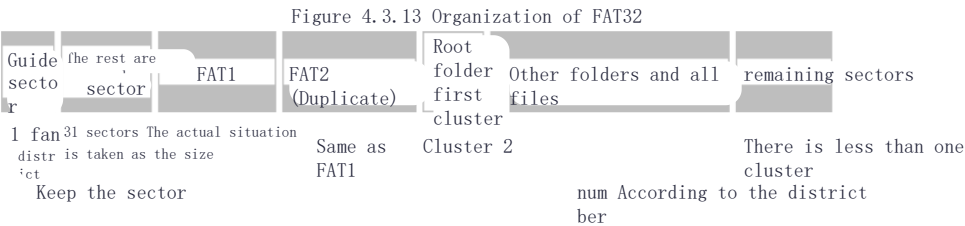
[\[Back to Index\]](#) ★FAT32

Storage Principle:

FAT32 is a very meritorious file system, and Microsoft has successfully designed and used it

Today, when NTFS hits overwhelmingly, FAT32 still occupies an important position in Microsoft's Windows file system. FAT32 was originally designed because of the shortcomings of FAT16 not supporting large partitions and the large capacity of the unit cluster, which led to a sharp waste of space. In practice, FAT32 is still successful.

The principle of FAT32 and FAT16 is basically the same, and Figure 4.3.13 shows the basic composition of FAT32 partitions.



[Back to Index]

FAT32 builds its DBR according to the characteristics of the partition during the formatting process, among which the BPB parameter is very important, you can go back and look at Tables 4 and 5. First, FAT32 retains the default number of sectors to 32, instead of just one for FAT16. This benefit is that it helps to extend the length of disk DBR instructions and allows for backup space for DBR sectors. As we have mentioned above, win98 or win2000 and winXP built on FAT32 have operating system boot code that does not only occupy a sector. Leaving extra reserved sectors can be used to expand the OS boot code. The backup sector number of the DBR sector is also recorded in the BPB. Backup sectors allow us to restore DBR in case of accidental disk corruption.

The data structure of the file allocation table in FAT32 is still the same as that of FAT16, except that FAT32 extends the number of binary bits of the record cluster chain to 32 bits, so this file system is called FAT32. The cluster chain of 32-bit binary bits determines that the FAT table can address up to 2T clusters. This way, even if the cluster is 1 sector in size, it is theoretically still able to address partitions in the 1TB range. However, in reality, FAT32 cannot address such a large space, and with the increase of partition space, the number of records in the FAT table will become bloated, which will seriously affect the performance of the system. So in practice, FAT32 partitions larger than 32GB are usually not formatted. WIN2000 OS and above no longer directly support formatting partitions over 32GB to FAT32, but WIN98 can still format FAT32 partitions as large as 127GB, but this is not necessary or recommended. At the same time, FAT32 also has a small limitation, FAT32 volumes must have at least 65527 clusters, so for small partitions, FAT16 or FAT12 still need to be used.

As the partition becomes larger, if the cluster is small, the file allocation table also grows. There will still be the above efficiency problems. It is necessary to read and write large files efficiently, and to reduce space waste as much as possible. FAT32 also specifies the size of clusters corresponding to the corresponding partition space, see Table 12:

Table 12 FAT32 partition size and pair factor cluster size

Partition space size for each cluster	size sectors	The cluster space is large small
<8GB	8	4k
>=8GB and < 16GB	168k	
>=16GB 且<32GB	32	16k
>=32GB	64	32k

[\[Back to Index\]](#)

The value meaning of the cluster is similar to FAT16, but the number of digits is a little longer, see Table 13 for comparison:

Table 13 Meaning of values for FAT system record items (16 rounds system)

The value of the FAT12 record	FAT16 record item value	FAT32 record item value	The performance of the corresponding cluster
000	0000	00000000	Unassigned clusters
002~FFF	0002~FFEF	00000002~FFFFFFEF	Assigned clusters
FF0~FF6	FFF0~FFF6	FFFFFFF0~FFFFFFF6	System retention
FF7	FFF7	FFFFFFF7	坏簇
FF8~FFF	FFF8~FFFF	FFFFFFF8~FFFFFFF	File end cluster

Another major overhaul in FAT32 is the documentation of root directories, i.e., equating root directories with ordinary files. In this way, the root directory does not have the limit of 512 directory items in FAT16, and when it is not enough, you can add a cluster chain and allocate empty clusters. Moreover, the location of the root directory is no longer rigidly fixed, and can be stored in any addressable cluster in the partition, although the root directory is usually the earliest directory table established (formatted and generated). So, what we see is basically the root first cluster occupying the first cluster in the cluster order. In Figure 4.3.12, it is also drawn according to this situation.

FAT32 clusters are still numbered as FAT16. The first cluster in the order is still numbered as the second cluster, which is usually used for the root directory (this is different from FAT16, the root directory of FAT16 does not occupy the cluster space, and the root directory of 32 sectors is the first cluster in the cluster area after that).

FAT32 has the same file addressing method as FAT16, but the meaning of the byte parameters of directory items is different from FAT16, on the one hand, it enables the directory item retention field in FAT16, and at the same time fully supports it

Long file name.
For directory items in short file formats. The meaning of its parameters is shown in Table 14: [Back to Index]

Table 14FAT32 Short File Directory Item 32 bytes of representation definition			
Byte	Offset (16)	Number of bytes	definition
	0x0~0x7	8	filename
	0x8~0xA	3	Extension: 00000000 (Read and Write) 00000001 (Read Only)
	0xB*	1	attri 00000010 (hidden) bute 00000100 (System) bytes 00001000 (Label) 00010000 (subdirectory) 00100000 (Archive)
	0xC	1	System retention
	0xD	1	10 milliseconds of creation time
	0xE~0xF	2	When the file was created
	0x10~0x11	2	The file was created on the date
	0x12~0x13	2	The date of last access to the file
	0x14~0x15	2	The file starts with a high 16 bits of the cluster number
	0x16~0x17	2	The most recent modification of the file
	0x18~0x19	2	The date of the most recent modification of the document
	0x1A~0x1B	2	The lower 16 bits of the file starting cluster number
	0x1C~0x1F	4	Indicates the length of the file

* This field cannot be valued at 0FH in the short file directory item, if the value is set to 0FH, the directory segment is the directory segment of the long file name

[Back to
Index] Note:

- (1) This is the meaning of the FAT32 short file format directory item. Its Chinese subject name, extension, time, and date algorithm is the same as FAT16.
- (2) Since FAT32 addressable cluster numbers reach 32-bit binary numbers. Therefore, the system also needs 32 bits to record the starting cluster address of the file (folder), and FAT32 enables the directory item offset 0x12~0x13 to indicate the higher 16 bits of the starting cluster number.
- (3) The file length is still expressed in 4 bytes, which means that FAT32 still only supports files (directories) smaller than 4GB, and files (directories) that exceed 4GB will be truncated and processed.

An important feature of FAT32 is full support for long file names. The long file name is still recorded in the directory item. In order for the lower version of the OS or program to read the long file name file correctly, the system automatically creates a corresponding short file name for all long file name files, so that the corresponding data can be searched by long file name

address, which can also be addressed with a short file name. An OS or program that does not support long file names ignores long file name fields that it deems unlawful, while an OS or program that supports long file names record and edit explicit items with long file names and hide short file names.

When creating a long file name file, the system will automatically add the corresponding short file name, and the general principles are:

(1) Add “~1” to the first 6 characters of the long file name to form a short file name, and the extension remains unchanged. (2) If this file name already exists, the number after the symbol “~” is incremented until 5. (3) If the number after “~” in the file name reaches 5, the short file name only uses the first two letters of the long file name. The last four letters of the short filename are generated by mathematically manipulating the remaining letters of the long filename, and then the suffix “~1” is added to the end (or other numbers if necessary to avoid duplicate filenames).

(4) If there are characters that cannot be read by the old OS or program, replace them with “_”

The implementation of long file names relies on a directory item offset to a byte of attribute of 0xB, which DOS and WIN32 consider illegal and ignore its existence when its attributes are: read-only, hidden, system, label, i.e. 0FH. This is the basis for the existence of long file names. Set the 0xB of directory items to 0F, and let the system define the rest, Windows 9x or Windows 2000, XP usually supports long file names of no more than 255 characters. Long file names are cut in 13-character increments, with each group occupying a directory item. Therefore, a file may need multiple directory items, and each directory item of the file name is arranged in reverse order in the directory table to prevent confusion with other file names.

Characters in long filenames are encoded in Unicode (a huge improvement), and each character takes up 2 bytes of space. The catalog items are defined in Table 15.

Table 15 FAT32 is a 32-byte representation definition of a long file directory item

Byte Offset (16)	Number of bytes	definition
0x0	7	Keep unused
	6	1 indicates the last directory item of the long file
	Attr ibut	5
	5	Keep unused
	e	4
	byte	4
	bit	3
	mean	2
	ing	2
		Sequence number value
0x1~0xA	1	
	0	
	10	Long file name unicode code(1)
	0xB	The long file name directory item flag is set to 0FH
	0xC	System retention
	0xD	Checksum (calculated from short file name)
	0xE~0x19	Long file name unicode code(2)
	0x1A~0x1B	File Start Cluster Number (currently standing 0)
	0x1C~0x1F	Long file name unicode code(3)
	4	

[\[Back to Index\]](#)

When storing long file names, the system always fills in the long file name directory items in reverse order, and then follows the corresponding short file name. As you can see from Table 15, the file start cluster, file size, and various time and date attributes of the corresponding file are not stored in the long file name. These attributes of the file are still stored in the short file name directory item, a long file name always corresponds to its corresponding short file name, short file name can be read without a long file name, but if the long file name does not have a corresponding short file name, no matter what system will ignore its existence. So short file names are crucial. Changing the filename and extension fields in short filenames (including deletion, as deletion is a rewrite of the first character E5H) in an environment where long filenames are not supported, renders long filenames useless. The connection between long and short file names is obviously not enough to maintain the location relationship between them. In fact, the checksum of 0xD bytes of long file names plays a very important role, which is obtained by using the 11 characters of short file names through an operation. The system determines whether the corresponding long file names and short file names match according to the corresponding algorithm. This algorithm is not easy to explain with a formula, so let's use a C program to illustrate it.

Suppose the filename has 11 characters that form the string `shortname[]`, and the checksum is represented by `chknum`. The process is as follows:

```
int i, j,chknum=0;
for (i=11; i>0; i--)
chksum = ((chksum & 1) ? 0x80 : 0) + (chksum >> 1)
+shortname[j++];
```

If the checksum calculated by the short file name is not equal to the data at the 0xD offset in the long file name. The system will not pair them anyway.

Based on the definition of directory items by long and short file names, plus the numbering and links of clusters, reading data on FAT32 is easy.

5. End. [\[Back to Index\]](#)

This article is from Data Recovery (www.sjhf.net), omissions are inevitable, I hope to correct it. Please retain this information if you need to reprint; If you need to make changes, please contact the author in the following ways

- 1、<http://www.sjhf.net>
- 2、zymail@vip.sina.com